Spotify Analysis

S2816575 **S2890878** **S2820897** **S2864694**

Abstract

This project analyses Spotify Top 200 chart data from 2017 to 2023 to understand what makes a track popular and to predict whether a song becomes a "hit" (enters the Top 20). The team cleans and aggregates song and artist level data, engineers audio, temporal, and artist features, and compares several classification models. Results show that adding temporal information to audio features, and to a lesser extent to artist history, yields fairly-strong predictive performance and provides useful signals on how sound, timing, and artist history jointly shape chart success.

1 Introduction

Streaming charts are now a key indicator of commercial success. However, it is still unclear which aspects of a track and its context most strongly influence whether it becomes a hit. There have been several machine learning approaches developed to predict music popularity. In the papers [1], [2], and [3], the authors applied various classifier methods on a dataset collected by Kaggle and extracted by Spotify to predict a song's success based on audio features. Some others, [4] and [5], define more diverse aspects of popularity or combine Youtube features of music videos in addition to audio features. In this report, we use Spotify Top 200 data and engineered audio, temporal, and artist features to model hit status as a binary classification problem. Initially, we experimented with regression models on continuous popularity scores, but we found that this setup did not align well with the practical question whether a track becomes a hit or not, and what features drive that prediction. This motivated us to reframe the problem as a binary classification task, which focusses directly on predicting hit status and avoids misleadingly good regression metrics that can arise from chart mechanics. By contrasting different feature views and classifiers, we aim to discover which features drive the popularity of a song. We explore the dataset, preprocess the data, and apply various machine learning techniques to identify key features that influence a song's popularity on Spotify. We begin by splitting the data into the following three feature-datasets: audio features, audio + temporal features and audio + artist features. Audio features contain the columns; "Danceability", "Energy", "Loudness", "Speechiness", "Acousticness", "Instrumentalness", "Valence". Temporal features include year, month and season, which is a derived variable from month. Thus the full list of temporal features are: "year", "month", "month_sin", "month_cos", "top10_season_index", "season_Spring", "season_Summer","season_Autumn", "season_Winter". Lastly, artist features are "genre", "artist" and "continent". We then perform exploratory data analysis to understand the distribution and relationships of these features with song popularity. Finally, we apply classification models to predict song popularity based on these features and evaluate their performance. The classification algorithms used are Logistic Regression, Random Forest, Neural Networks and Support Vector Machines. Beyond prior work, which often focuses either on audio content alone or a single modelling approach, our study adds a systematically cleaned song–date and artist-level dataset, a side-by-side comparison of multiple feature views (audio, audio+temporal, and audio+artist) across four classifiers, and a richer evaluation using accuracy, precision, recall, F1 and AUC. This design allows us not only to benchmark standard methods on a large real-world chart dataset, but also to highlight where temporal context and artist history provide additional signal over raw audio, and to point toward future extensions such as ordinal (Top-10/Top-50) prediction and sequence models that track how songs rise and fall over time.

2 Data Preparation

2.1 Data Cleaning and aggregation

The dataset contains data quality issues stemming from two primary types of structural duplication, leading to the need for aggregation before analysis. First, there is a collaboration duplicity, where a single track's performance on a given day is split into multiple rows, one for each collaborating artist, causing features such as the song's total points and track features to be redundantly listed. Second, there are conflicting entries where a single song identified by its unique ID and Date appears multiple times with different ranks, while all other features remain identical. To resolve these issues, the dataset was aggregated at the song-date level, ensuring that each song on a given date was represented by a single consolidated record rather than multiple overlapping rows. For cases where inconsistent ranks exist, the lowest rank value is retained. In addition to the audio features aggregation, a temporal and artist-level dataset, along with audio features, was created to understand the most important feature set.

3 Exploratory Data Analysis

The dataset comprises 651,936 chart entries with 20 columns, covering 7,457 distinct tracks performed by 2,208 featured artists. For our models, we have defined a song as a hit or popular (the target column) if it ranks within the top 20 on the charts at any time during its chart life. This makes the task a *binary classification* task. For **audio features only**, we first check for linear trends by analysing the correlation between the audio features. Using a heatmap, we analyse whether these features exhibit low linear correlation with the target column and amongst themselves. The highest correlation observed was around 0.40 between "Danceability" and "Valence", which aligns with the intuitive notion that more danceable songs often carry a more positive or upbeat nuance. We plotted the distributions for the audio features and observed that most columns lay within the range of 0 to 1; however, Loudness exhibited negative values ranging from -34475 to 1509, which were much larger than those of the other audio features. To bring them to a common scale for the fair training of the models, we applied zero mean and unit variance normalisation. The mean and standard deviation were computed only on the training data, and then the same parameters were applied to the validation and test sets to prevent data leakage. The equation below describes the normalization:

$$z = \frac{(x - \mu)}{\sigma}$$

During the exploration of **temporal and audio features** , we extracted features such as month, year, and week from the 'Date' column to check for seasonal and temporal patterns in the dataset. Plots of popularity across months showed a decent relation with popularity, thus to capture seasonal patterns in the data we applied a cyclic patten to the month column extracted from 'Date'. Since months and days are periodic, they can be transformed into a circular structure year wise using sine and cosine encodings. We also examined whether the characteristics of popular songs varied over time, through the line plots of yearly averages for each audio feature. In Figure 1 it is indicated that average level of "Danceability" and "Speechiness" of the songs were generally higher in earlier years(2017–2020) but have declined since then, while Energy and Loudness level decreased to their minimum point at year 2020, then started to gradually recover. On the other hand, Instrumentalness and Acousticness is showing a mostly constant rise in trend over the years, suggesting that songs with these features are growing popularity as years passed.
To analyze seasonal effects, we visualized average value of audio features across seasons using a heatmap. The results, depicted in figure (Figure 2), indicate that the average levels of Danceability and Energy are relatively higher in summer, while Acousticness is higher in autumn and winter. This can be interpreted as the tendency for more upbeat music to be consumed during summer, and calmer music to be preferred in the winter which also aligns with common intuition. Finally, when visualising **artist level features (and audio)** , we observe that the most popular tracks typically feature one or two artists. On analyzing individual points of artists we see that Bad Bunny, Ed Sheeran and Post Malone are the highest scoring artists. Other categorical features, such as Nationality and Continent, show some variation overall, with countries like Thailand, Senegal, and Lithuania having slightly higher average Points (Total). To include impact of artist we have applied target encoding [6]. For each unique artist in the training set we computed mean target value, which gives us a metric for how often a particular artist produces popular songs.

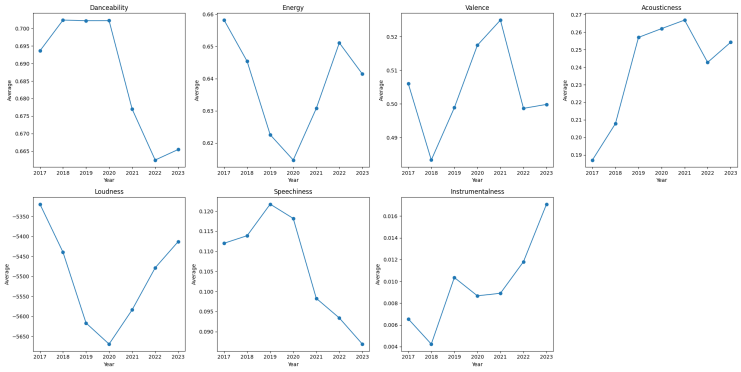

Figure 1: Yearly trends of average values for each audio feature.

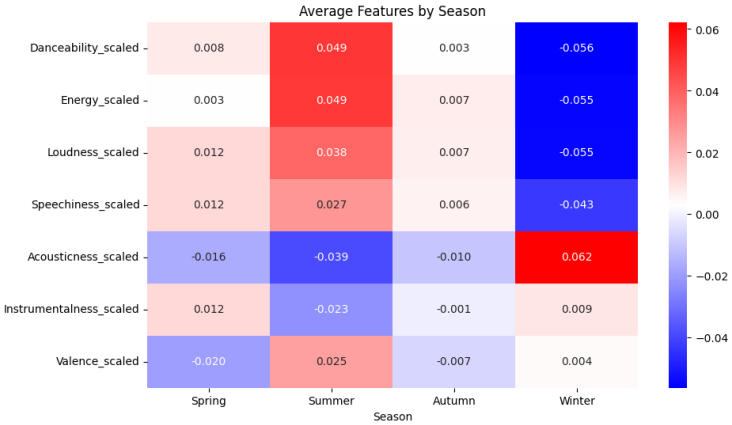

Figure 2: Heatmap of average audio feature values across seasons.

4 Task: What features drive the popularity of a song?

4.1 Motivation

We initially tried several regression models (Linear Regression, Gradient Boosting, LightGBM, Random Forest) using total points as the target, and although some achieved strong numerical scores, they did not answer the question the business cares about: whether a track will be popular (hit top-20 charts), or not. Because points are mechanically linked to the chart construction, the regression setup was also quite fragile—small leaks of rank like information could make the task almost trivial, giving overly optimistic metrics that did not translate into better decisions. In addition, regression losses treat every error equally, even though a small mistake around the Top 20 cut off is much more important than a large error for a track that is nowhere near charting. By switching to a classification task, we align the target with the actual decision, obtain calibrated probabilities that a song will reach the Top 20, and can explicitly handle class imbalance and threshold choice using metrics such as precision, recall, F1 and AUC. Overall, the classification framework is better matched to the problem, avoids misleadingly "good" regression scores, and produces outputs that are easier for the team to interpret and act upon.

4.2 Classification algorithms

We apply four different classification algorithms to each dataset and evaluate the performance across each split/model combination. **Logistic Regression (LR)** will be used as a linear baseline. For an input feature vector $\boldsymbol{x} \in \mathbb{R}^D$ and an augmented feature map $\Phi(\boldsymbol{x}) = [1, x_1, \ldots, x_D]^\top$, LR assumes

$$f(\boldsymbol{x}; \boldsymbol{w}) = \boldsymbol{w}^\top \Phi(\boldsymbol{x}), \qquad p(y = 1 \mid \boldsymbol{x}) = \sigma(f(\boldsymbol{x}; \boldsymbol{w})) = \frac{1}{1 + e^{-f(\boldsymbol{x}; \boldsymbol{w})}},$$

with $y \in \{0, 1\}$ indicating whether the track is a hit. Predictions use a fixed threshold $\tau = 0.5$: we predict "hit" if $p(y = 1 \mid \boldsymbol{x}) > 0.5$. The parameters $\boldsymbol{w}$ are learnt by maximising the (Bernoulli) likelihood of the data, which in practice corresponds to minimising the cross-entropy loss under the usual i.i.d. assumption[1]. **Random Forest (RF)** is an ensemble learning method that constructs multiple decision trees during training and outputs the mode of the classes (classification) of the individual trees. Each tree is built using a random subset of the training data and a random subset of features, which helps to reduce overfitting and improve generalization. A **neural network (NN)** is a powerful classification algorithm that can model complex non-linear relationships between input features and target classes [7]. Given input $\boldsymbol{x}$, a simple one-hidden-layer network can be expressed as

$$\boldsymbol{h} = \sigma(\boldsymbol{W}_1 \boldsymbol{x} + \boldsymbol{b}_1),$$
$$\hat{y} = \sigma(\boldsymbol{W}_2 \boldsymbol{h} + \boldsymbol{b}_2),$$

where σ denotes a non-linear activation, $\boldsymbol{h}$ is the hidden representation, and $\hat{y}$ approximates $p(y = 1 \mid \boldsymbol{x})$. Training minimises the cross-entropy loss over the dataset. Figure **??** illustrates the architecture of a feed-forward neural network used for our classification task. **Support Vector Machines (SVMs)** aim to find a decision boundary with maximum margin between positive and negative examples. In the linear case, this can be written as

$$\min_{\boldsymbol{w}, b} \quad \frac{1}{2}\|\boldsymbol{w}\|^2 \text{ s.t.} \qquad\qquad y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) \geq 1 \quad \text{for all } i,$$

with soft-margin and kernel extensions for non-separable and non-linear data. [2] Training an exact RBF SVM on the full Spotify dataset is computationally expensive. It requires constructing the full kernel matrix. To avoid this, we approximate the RBF kernel using a Nyström [8] feature map and then train a linear SVM.[3].

4.3 Metrics

To analyse our experiment results, we used a series of metrics to evaluate each model performance. The first metric is **accuracy**, which is calculated by the proportion of the number of correct predictions to the total predictions made. Accuracy was first taken into account, as it is one of the most commonly used metrics for evaluating binary classification tasks[9]. However, in cases of class imbalances, accuracy may give misleading results; therefore, we decided to additionally utilise **precision** and **recall** to mitigate class imbalance issues. These metrics can be especially useful for our task, as our output classes are imbalanced due to the nature of our definition of 'popular'. Precision, or Positive Predictive Rate(PPR), is the ratio of true positive predictions out of all positive predictions. On the other hand recall, also known as True Positive Rate(TPR), is the proportion of true positive predictions out of all actual positive cases. Precision and recall can be used as pairs because they have an inverse relationship where improving one metric leads to weakening the other[10]. This leads to our next choice for evaluation, the **F1 score**. The F-measure is the harmonic mean of precision(Pr) and recall(Re). The value, specifically F1 score, can be obtained by the following equation:

$$F1 = \frac{2 \times \text{Pr} \times \text{Re}}{\text{Pr} + \text{Re}}$$

Although the F-score is a balanced metric based on precision and recall, it still has the limitation that the score can vary depending on the classification threshold. This issue does not affect our current task, since we use a fixed threshold to determine whether a song is successful or not. However, to evaluate the model's general classification performance in a way that is applicable to different scenarios, we also consider another metric introduced below: the AUC score. The **AUC score** is obtained by calculating the area under ROC curve, i.e. Receiver Operating Characteristic curve, which is a plot showing model performances across all possible thresholds. An ideal model will have the AUC score of 1, whereas a model that randomly predicts classes will have the AUC score of 0.5, plotting its visual representation of the performance into a diagonal line connecting $(0, 0)$ to $(1, 1)$.

[1]For a dataset $\mathfrak{D} = \{(\boldsymbol{x}_n, y_n)\}_{n=1}^N$, the likelihood can be written as $\prod_{n=1}^N p(y_n \mid \boldsymbol{x}_n; \boldsymbol{w})$ with $p(y_n \mid \boldsymbol{x}_n; \boldsymbol{w}) = p(y = 1 \mid \boldsymbol{x}_n; \boldsymbol{w})^{y_n}(1 - p(y = 1 \mid \boldsymbol{x}_n; \boldsymbol{w}))^{1-y_n}$.

[2]We use the standard hinge-loss formulation with slack variables and an RBF kernel in practice.

[3]The full model, therefore, is `StandardScaler` → `Nystroem(kernel="rbf", n_components=600)` → `SGDClassifier`.

4.4 Training

We first partition the dataset into 75:25 cross-validation and holdout datsets[4]. We apply early stopping when validation loss fails to improve for two epochs, and reserve the 25% holdout set exclusively for the final evaluation. Below we summarise the hyperparameters selected after the dedicated tuning notebooks [5]. We train an `SGDClassifier` with the logistic loss so we can capture epoch-level histories. The best configuration uses a relatively aggressive elastic-net penalty: $\alpha = 0.01$, `penalty=elasticnet`, `l1_ratio` $= 0.3$, `learning_rate=constant` with $\eta_0 = 0.01$, `power_t` $= 0.5$. For Neural Networks, the cross-validated sweep highlighted a compact single hidden layer as the most reliable option. We therefore adopt `MLPClassifier` with hidden layer sizes (32), `activation=tanh`, L_2 penalty $\alpha = 10^{-4}$, `learning_rate=adaptive` with `learning_rate_init` $= 10^{-2}$, `momentum`=0.9, `solver=sgd`. For the Random Forest, the warm-start grid search varied tree counts, depth caps, leaf sizes, feature subsampling, and bootstrap usage. The winning configuration trains 600 trees with depth capped at 20, `min_samples_leaf` $= 1$, `max_features` $=$ `sqrt`, and `bootstrap=True`. We keep `class_weight=balanced`, `n_jobs`$=-1$, and `warm_start=True` so that we can add trees in stages (roughly 60 per epoch) and record the incremental loss/accuracy curves. Exact RBF SVMs are prohibitively expensive at our scale, as mentioned in section 4.2. The final model sets `approximate=True`, `n_components`=100, `kernel=rbf`, `gamma=scale`, and feeds the transformed features into an `SGDClassifier` with hinge loss, $\alpha = 5 \times 10^{-5}$, and `max_iter`=100. This preserves a non-linear decision boundary while keeping training incremental and memory efficient.

5 Results

Figure 3 summarises how each model behaves on the holdout(test) split when the feature space expands. Several trends emerge. Figure 3 confirm that acoustic descriptors on their own carry limited discriminative power. The holdout results illustrate the challenge: random forest attains accuracy 0.86 but only 0.18 precision, 0.11 recall, and F1 0.13; the SVM variant reaches a higher recall of 0.36 yet slips to a precision of 0.10 (F1 0.15). Logistic regression hovers around chance (accuracy 0.50, AUC 0.48) and the neural network collapses to precision 0.04. These numbers show that, despite acceptable accuracy driven by the class prior, audio-only models still misidentify most Top-20 tracks. Artist metadata (collaboration counts, nationality indicators, target encoding) improves cross-validation scores but provides modest gains on the held-out slice. Random forest reaches accuracy 0.84 yet plateaus at precision 0.14, recall 0.18, and F1 0.18, indicating that the features help ranking more than absolute classification. SVM (Approx RBF) retains the best recall in this setting (0.43) but still couples it with precision 0.14. Logistic regression and the neural network trail with F1 below 0.20. These results suggest that artist context adds incremental signal but, without temporal cues, does not fully resolve the imbalance challenge. Adding calendar information (year/month plus sine/cosine encodings) produces the largest lift across metrics. On the holdout split the random forest now balances precision 0.40 with recall 0.79 (F1 0.53, AUC 0.85), providing a practical classifier that retrieves most hits while keeping false alarms manageable. The neural network mirrors this trend with precision 0.59, recall 0.28, and AUC 0.86, while the SVM pushes recall to 0.83 (precision 0.31, F1 0.46, AUC 0.85). Even logistic regression benefits, achieving F1 0.45 and AUC 0.80. Temporal context therefore supplies the clearest signal for identifying future Top-20 entries. To see the results across all dataset splits, as well as the results for both training (cross-validation) and testing (holdout) runs, see appendix B.

6 Conclusions

Amongst all three feature sets, the audio + temporal configuration paired with either Random Forest (F1 0.53, AUC 0.85) or SVM (recall 0.83, AUC 0.85) delivered the best overall performance. Models using only audio features struggled to distinguish hits, exhibiting low precision and recall. Adding

[4]Within the cross-validation portion we carve out a secondary validation slice using a 20% share so that incremental models can monitor dev loss/AUC while still keeping the majority of data for fitting. During training we log per-epoch loss, accuracy, and AUC for both the cross-train and cross-val subsets.

[5]See `logreg_hyperparameter_tuning.ipynb`, `NN_hyperparameter_tuning.ipynb`, `random_forest_hyperparameter_tuning.ipynb`, and `svm_hyperparameter_tuning.ipynb`

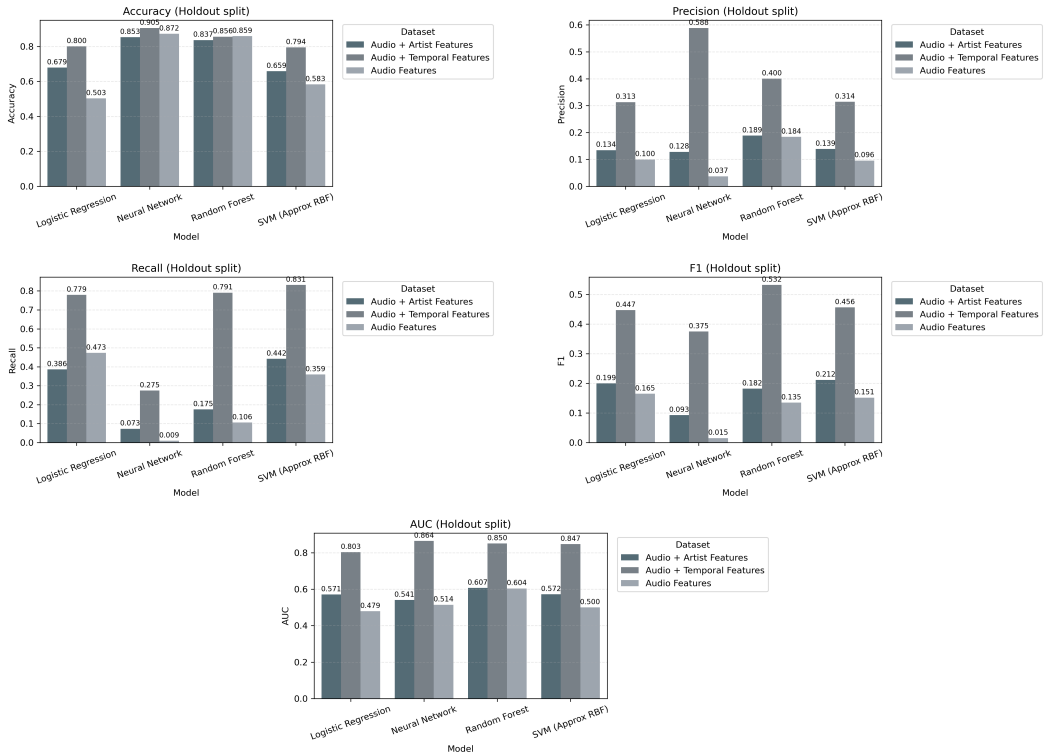

Figure 3: Test-split performance per metric across feature sets (Audio, Temporal+Audio, Audio+Artist).

artist features slightly improved all the metrics; however, audio + temporal data boosted performance the most. These results emphasise that timing signals and non-linear models are critical for flagging future Top-20 tracks. The results show that predicting whether a song on Spotify will become popular hinges on when a track is released, to an extent who performs it, and not merely on how it sounds. Our classification framing with balanced metrics such as F1 and AUC captures that nuance better than regression on chart position. The remaining false positives and negatives primarily arise from data imbalance and missing context, suggesting that incorporating additional temporal data or contextual information could improve the model's ability to identify chart hits. Next steps include exploring multi-class or ordinal ranking (e.g., predicting Top-10, Top-50, etc.), using richer artist networks or textual content, and testing sequential models to better capture track momentum. We could also refine probability calibration, expand features to include playlist exposure or social trends, and apply the approach to new release scenarios for more actionable predictions.

Appendix

A Further EDA Analysis

As an extension to the EDA documented above, our team conducted a focused outlier and distribution assessment across all numeric features. We produced horizontal boxplots and complementary histograms to characterise spread, skew, and extreme values, then performed a train-only winsorisation check to determine whether conservative clipping would improve robustness without distorting genuine signal. This exercise primarily targeted the loudness variable, where a small number of highly negative dB readings risked undue influence on scale-sensitive methods. Post-check, the bounded audio features (e.g., danceability, energy, valence) retained their expected shapes, while loudness extremes were flagged for clipping using thresholds learned exclusively from the training window to avoid temporal leakage.

In parallel, the team examined the relationship between the features and the target to uncover non-linear patterns that simple correlations might miss. For each audio attribute, we generated scatter plots against total points overlaid with LOWESS smooths, revealing gentle positive associations for dance-ability and valence, a mild inverted-U for energy, and diminishing returns for acoustic-ness and instrumental-ness. We complemented these visuals with a broader correlation heatmap that included rank and individual points, as well as aggregate bar charts of total points by weekday and by month. Together, these analyses indicated that audio features exert modest linear effects but meaningful non-linear trends, and that temporal aggregates are broadly flat by weekday with only mild monthly seasonality—findings that collectively support the use of non-linear, time-aware modeling in subsequent stages.

A.1 Pearson correlation analysis

For the exploratory analysis, our team first looked at the boxplots of the scaled features and conducted a correlation analysis of the seven audio features and visualized the results in a heatmap. From Figure 4, we observed that most correlations between the seven features were relatively weak, suggesting that the features tend to behave independently. The highest correlation was around 0.40 between Danceability and Valence, which aligns with the intuitive notion that more danceable songs often carry a more positive or upbeat nuance.

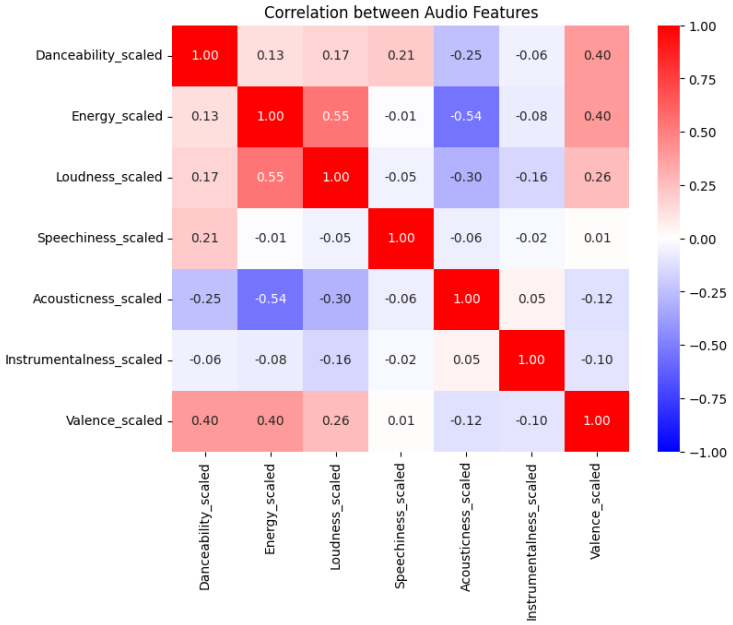

Figure 4: Heatmap of correlations between the seven audio features.

B Evaluation Results Detailed

Feature set	Metric	Logistic Regression	Random Forest	Neural Network	SVM (Approx RBF)
Audio + Artist Features					
	Cross-Validation Accuracy	0.682	0.867	0.924	0.657
	Cross-Validation Precision	0.189	0.416	0.724	0.184
	Cross-Validation Recall	0.711	0.978	0.327	0.761
	Cross-Validation F1 Score	0.298	0.584	0.451	0.297
	Cross-Validation Auc	0.772	0.970	0.909	0.771
	Holdout Accuracy	0.679	0.837	0.853	0.659
	Holdout Precision	0.134	0.189	0.128	0.139
	Holdout Recall	0.386	0.175	0.073	0.442
	Holdout F1 Score	0.199	0.182	0.093	0.212
	Holdout Auc	0.571	0.607	0.541	0.572
Audio + Temporal Features					
	Cross-Validation Accuracy	0.798	0.846	0.915	0.776
	Cross-Validation Precision	0.288	0.366	0.596	0.280
	Cross-Validation Recall	0.764	0.843	0.310	0.864
	Cross-Validation F1 Score	0.419	0.510	0.408	0.423
	Cross-Validation Auc	0.815	0.901	0.889	0.850
	Holdout Accuracy	0.800	0.856	0.905	0.794
	Holdout Precision	0.313	0.400	0.588	0.314
	Holdout Recall	0.779	0.791	0.275	0.831
	Holdout F1 Score	0.447	0.532	0.375	0.456
	Holdout Auc	0.803	0.850	0.864	0.847
Audio Features					
	Cross-Validation Accuracy	0.507	0.757	0.907	0.634
	Cross-Validation Precision	0.109	0.273	0.697	0.132
	Cross-Validation Recall	0.587	0.936	0.040	0.512
	Cross-Validation F1 Score	0.184	0.422	0.075	0.210
	Cross-Validation Auc	0.559	0.905	0.787	0.629
	Holdout Accuracy	0.503	0.859	0.872	0.583
	Holdout Precision	0.100	0.184	0.037	0.096
	Holdout Recall	0.473	0.106	0.009	0.359
	Holdout F1 Score	0.165	0.135	0.015	0.151
	Holdout Auc	0.479	0.604	0.514	0.500

Table 1: Cross-validation and holdout metrics across feature sets

Statement of Contribution

- S2820897 - I firstly started with the EDA and finding correlation on the audio-only branch of the dataset. Furthermore, I explored different regression models (Linear Regression, Gradient Boosting, Random Forest and LightGBM) on the cleaned audio features, tuned their settings, and compared their performance to understand how well continuous popularity could be explained just from how a track sounds.
- S2816575 - I concentrated on the classification side for the audio-only dataset. I prepared the feature matrices, trained several classifiers, analysed metrics such as accuracy, F1 and AUC, and produced the summary tables and plots that we later used to discuss what the audio features alone can and cannot capture about hit status.
- S2864694 - I led the audio+temporal route after performing EDA on temporal influence on chart flow. I engineered calendar and seasonal variables, combined them with audio features, and then trained multiple classification models on this richer view. I was mainly responsible for interpreting how timing (year, month, season) interacts with audio characteristics in driving chart outcomes.
- S2890878 - I worked on the artist+audio branch, along-with performing EDA and FE on the complete data. I created the artist-level features, joined them with track audio descriptors,and ran several classification models on this dataset. I also helped align the three strands by comparing results across the audio-only,audio+temporal and audio+artist experiments.
- This project was sure a very much joint effort. Although we worked in isolation for getting our own findings, we met regularly and kept a track to compare results, question each other's assumptions and refine the experiments. Henceforth, the final report has been a collaborative effort, with everyone contributing their own findings and checking the others' sections so that the story of the project reflected the whole team's work and led to a coherent, successful outcome.

Use of Generative AI

For this coursework we made limited, tool-style use of generative AI. We mainly used an LLM to help draft small utility scripts for plotting model results, populating metric tables from our outputs, and sketching example code for trying different hyperparameter combinations, which we then adapted, debugged and integrated ourselves. We also used it to point us towards relevant past work on hit-song prediction and Spotify-based studies, treating it as a way to find papers and links more quickly before reading the sources directly. All analysis decisions, model implementations and the final wording of the report were done by the group, with AI used only as a support tool rather than as a replacement for our own work.

References

[1] Dr Sharma, Dr Pareek, Mr Pathak, and Ms Sakariya. Predicting music popularity using machine learning algorithm and music metrics available in spotify. *JOURNAL OF DEVELOPMENT ECONOMICS AND MANAGEMENT RESEARCH STUDIES*, 09:10–19, 01 2022.

[2] Joshua S. Gulmatico, Julie Ann B. Susa, Mon Arjay F. Malbog, Aimee Acoba, Marte D. Nipas, and Jennalyn N. Mindoro. Spotipred: A machine learning approach prediction of spotify music popularity by audio features. In *2022 Second International Conference on Power, Control and Computing Technologies (ICPC2T)*, pages 1–5, 2022.

[3] Kai Middlebrook and Kian Sheik. Song hit prediction: Predicting billboard hits using spotify data. *CoRR*, abs/1908.08609, 2019.

[4] Junghyuk Lee and Jong-Seok Lee. Music popularity: Metrics, characteristics, and audio-based prediction. *IEEE Transactions on Multimedia*, 20(11):3173–3182, 2018.

[5] Raheem M Yee YK. Predicting music popularity using spotify and youtube features. *Indian Journal of Science and Technology*, 15(36):1786–1799, 2022.

[6] Dan Micci-Barreca. A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. *ACM SIGKDD Explorations Newsletter*, 3(1):27–32, 2001.

[7] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd edition, 2025. Online manuscript released August 24, 2025.

[8] T. Leen, T. Dietterich, Volker Tresp, Christopher Williams, and Matthias Seeger. Using the nystroem method to speed up kernel machines. 01 2000.

[9] Oona Rainio, Jarmo Teuho, and Riku Klén. Evaluation metrics and statistical tests for machine learning. *Scientific Reports*, 14:6086, 2024.

[10] Google Developers. Classification: Accuracy, precision and recall. `https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall`. Accessed: 17 November 2025.